	Course Name: Advanced Web Technology	EXPERIMENT NO. 11
	Course Code: 20CP314P	Branch: CSE
	Faculty: Komal Singh	Semester: VI
Submitted by: Harsh Shah Roll no: 21BCP359		

Objective: Implement a web-app using Flask framework

1. Design a 'Hello World' Web-application using Flask framework

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
def hello_world():
```

```
    return 'Hello, World!'
```

```
if __name__ == '__main__':
```

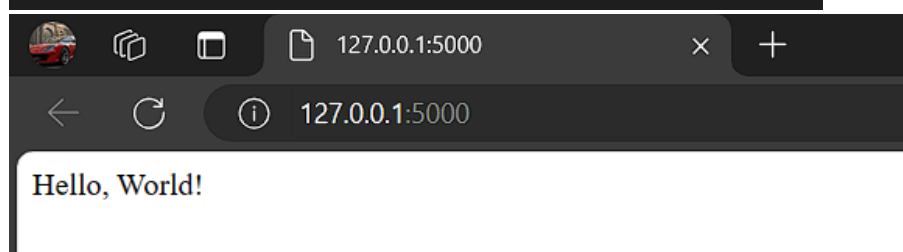
```
    app.run(debug=True)
```

```

PROBLEMS 7 TERMINAL OUTPUT PORTS DEBUG CONSOLE
PS D:\SEM-6\Advanced Web Technology\Lab_11> pip install flask
Requirement already satisfied: flask in c:\python39\lib\site-packages (2.0.1)
Requirement already satisfied: Werkzeug>=2.0 in c:\python39\lib\site-packages (from flask) (2.0.1)
Requirement already satisfied: Jinja2>=3.0 in c:\python39\lib\site-packages (from flask) (3.0.1)
Requirement already satisfied: itsdangerous>=2.0 in c:\python39\lib\site-packages (from flask) (2.0.1)
Requirement already satisfied: click>=7.1.2 in c:\python39\lib\site-packages (from flask) (8.0.1)
Requirement already satisfied: colorama in c:\python39\lib\site-packages (from click>=7.1.2->flask) (0.4.4)
Requirement already satisfied: MarkupSafe>=2.0 in c:\python39\lib\site-packages (from Jinja2>=3.0->flask) (2.0.1)

[notice] A new release of pip is available: 23.3.1 -> 24.0
[notice] To update, run: python.exe -m pip install --upgrade pip
PS D:\SEM-6\Advanced Web Technology\Lab_11> python app.py
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Restarting with stat
* Debugger is active!
* Debugger PIN: 868-049-390
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
127.0.0.1 - - [25/Apr/2024 22:23:59] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [25/Apr/2024 22:23:59] "GET /favicon.ico HTTP/1.1" 404 -

```



2. Connect the Flask Web-Application using SQLAlchemy to PostgreSQL and perform CRUD commands.

app.py

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://postgres:password@localhost/user2'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), nullable=False)
    email = db.Column(db.String(120), nullable=False)

    def __repr__(self):
        return f'<User {self.username}>'

@app.route('/users', methods=['GET'])
def get_users():
    users = User.query.all()
    return jsonify([{'id': user.id, 'username': user.username, 'email': user.email} for user in users])

@app.route('/users', methods=['POST'])
def create_user():
    data = request.json
    new_user = User(username=data['username'], email=data['email'])
    db.session.add(new_user)
    db.session.commit()
    return jsonify({'message': 'User created successfully'})
```

```

@app.route('/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    user = User.query.get_or_404(user_id)

    data = request.json

    user.username = data.get('username', user.username)
    user.email = data.get('email', user.email)

    db.session.commit()

    return jsonify({'message': 'User updated successfully'})

@app.route('/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    user = User.query.get_or_404(user_id)

    db.session.delete(user)

    db.session.commit()

    return jsonify({'message': 'User deleted successfully'})

if __name__ == '__main__':
    app.run(debug=True)

```

```

PS D:\SEM-6\Advanced Web Technology\flask_crud> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 868-049-390

```

Create

```

PS D:\SEM-6\Advanced Web Technology\flask_crud> Invoke-RestMethod -Uri "http://localhost:5000/users" -Method Post -ContentType "application/json" -Body '{
>>   "username": "new_user",
>>   "email": "new_user@example.com"
>> }'
>>

message
-----
User created successfully

```

Update

```

PS D:\SEM-6\Advanced Web Technology\flask_crud> Invoke-RestMethod -Uri "http://localhost:5000/users/1" -Method Put -ContentType "application/json" -Body '{
>>   "username": "updated_username",
>>   "email": "updated_email@example.com"
>> }'
>>

message
-----
User updated successfully

```

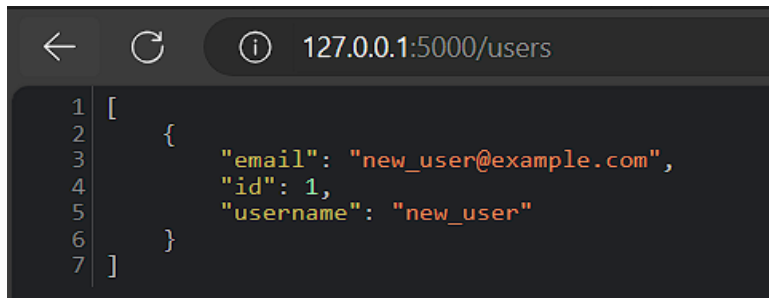
Delete

```
PS D:\SEM-6\Advanced Web Technology\flask_crud> Invoke-RestMethod -Uri "http://localhost:5000/users/1" -Method Delete -ContentType "application/json" -Body '{
>>   "username": "updated_username",
>>   "email": "updated_email@example.com"
>> }'
>>

message
-----
User deleted successfully
```

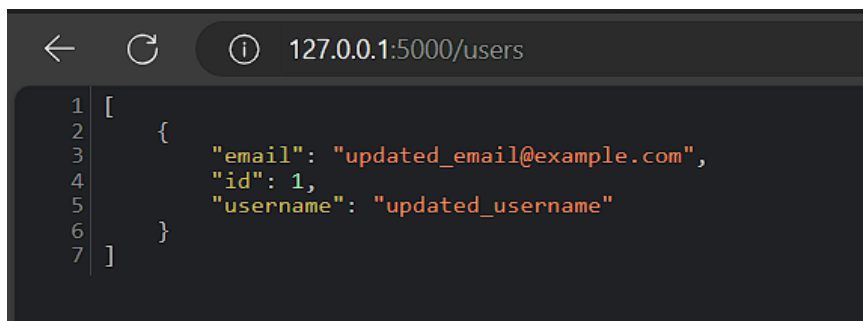
Output:

Create



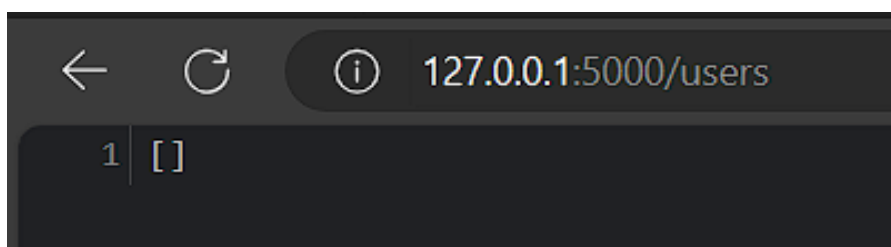
```
1 [
2   {
3     "email": "new_user@example.com",
4     "id": 1,
5     "username": "new_user"
6   }
7 ]
```

Update



```
1 [
2   {
3     "email": "updated_email@example.com",
4     "id": 1,
5     "username": "updated_username"
6   }
7 ]
```

Delete



```
1 []
```